

cluster revision.

For example: If a new product client application supporting revision 3 of cluster X wishes to take advantage of the new behavior that is mandated by revision 3, then the application can read the revision of the corresponding server cluster X in each remote endpoint. If a corresponding cluster X supports revision 3 or greater, then the behavior is supported.

Examples of changes to a cluster that require incrementing the revision:

- Changing the behavior of the cluster
- Changing a read only attribute to become writable
- Adding new attributes (e.g. min and max of an existing attribute value)
- Adding new commands, actions, or behavior
- Adding one or more fields to an existing command
- Adding a new enumerated value to an attribute
- Adding qualities to an existing attribute (e.g. Q)
- Changing anything that is optional to mandatory
- Changing dependencies of optional items
- Deprecating parts of the cluster specification
- Any non-editorial specification text change

7.10.2. Cluster Optional Features

In general, as the number of optional elements in a cluster specification increases, the number of possible combinations increases, which could decrease the interoperability of that cluster.

Each cluster has a mandatory feature set that consists of mandatory elements such as attributes, commands, fields, values, dependencies, behavior, etc.

A cluster specification MAY have optional feature sets, each supported by a set of elements (see [Section 7.13.2, “FeatureMap”](#)).

There is no requirement that each cluster instance supports the same set of optional elements.

If an application knows the [ClusterRevision](#) and [FeatureMap](#) supported by a cluster instance, then it knows the exact specification text required to be implemented by that instance.

7.10.3. Cluster Data Version

A cluster data version is a metadata increment-only counter value, maintained for each cluster instance. A cluster data version represents an exact & coherent state of cluster attribute data at present. An application may externally hold a data version (called a held data version) published by a cluster instance which then represents a cluster instance state at some time in the past. An application may use a held data version to optimize future interactions, by indicating the held data ver-